

UNIVERSIDAD DE GUANAJUATO



Programación en Ingenierías

Manipulación de bits

La manipulación de bits u operadores bit a bit (bitwise) son operadores comunes en aplicaciones embebidas, ya que nos permiten configurar los registros para usar el hardware incluido, acceder a los puertos de entradas y salida, hacer cálculos rápidos, verificar la autenticidad de los datos enviados/recibidos, entre otras aplicaciones.

Los operadores bit a bit que permiten una manipulación binaria, son los mismos operadores que para el álgebra booleana:

Operador	A	B	C	
<i>AND</i> (&)	$12_{10} = 1100_2$	$10_{10} = 1010_2$	$C = A \& B$	$8_{10} = 1000_2$
<i>OR</i> ()	$12_{10} = 1100_2$	$10_{10} = 1010_2$	$C = A B$	$14_{10} = 1110_2$
<i>NOT</i> (~)	$12_{10} = 1100_2$		$C = \sim A$	$3_{10} = 0011_2$
<i>NOT</i> (~)		$10_{10} = 1010_2$	$C = \sim B$	$5_{10} = 0101_2$
<i>XOR</i> (^)	$12_{10} = 1100_2$	$10_{10} = 1010_2$	$C = A \wedge B$	$6_{10} = 0110_2$

En el lenguaje C existen adicionalmente los operadores de corrimiento

Operador	A	n	$B = A \gg n$	B
<i>derecha</i> ($\gg$)	$45_{10} = 00101101_2$	1	$B = A \gg 1$	$22_{10} = 00010110_2$
<i>derecha</i> ($\gg$)	$45_{10} = 00101101_2$	2	$B = A \gg 2$	$11_{10} = 00001011_2$
<i>derecha</i> ($\gg$)	$45_{10} = 00101101_2$	3	$B = A \gg 3$	$5_{10} = 00000101_2$
<i>izquierda</i> ($\ll$)	$45_{10} = 00101101_2$	1	$B = A \ll 1$	$90_{10} = 01011010_2$
<i>izquierda</i> ($\ll$)	$45_{10} = 00101101_2$	2	$B = A \ll 2$	$180_{10} = 10110100_2$
<i>izquierda</i> ($\ll$)	$45_{10} = 00101101_2$	3	$B = A \ll 3$	$104_{10} = 01101000_2$

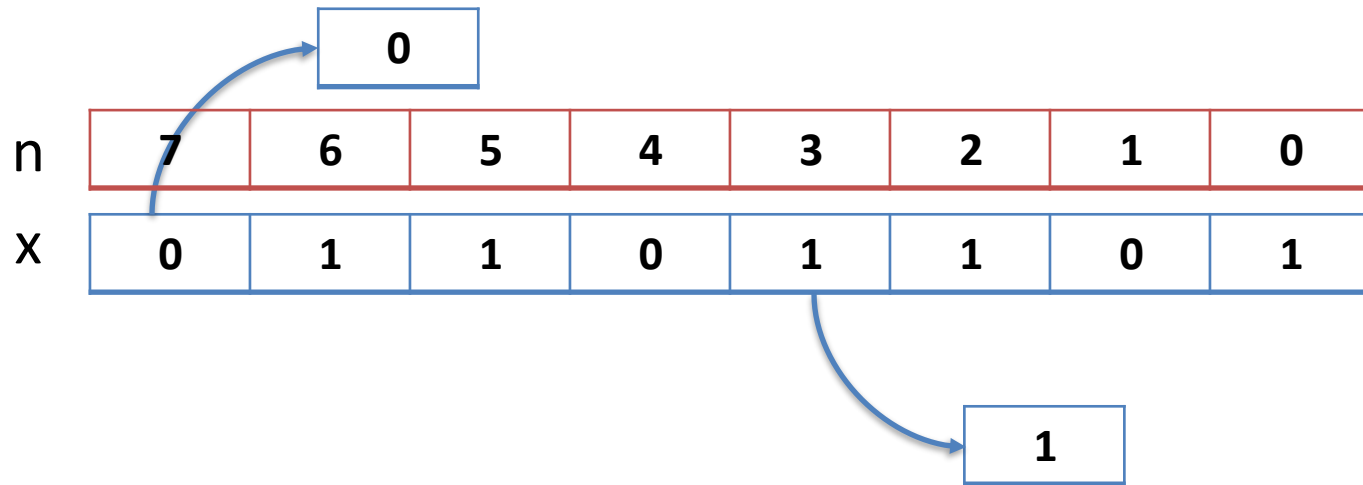


Los operadores básicos con bit son:

1. Obtener el n-ésimo bit de una variable (`bit_get`).
2. Establecer a '1' el n-ésimo bit de una variable (`bit_set`).
3. Establecer a '0' el n-ésimo bit de una variable (`bit_clear`).
4. Invertir el valor del n-ésimo bit de una variable (`bit_toggle`).
5. Establecer el valor 'v' en el n-ésimo bit de una variable (`bit_write`).

1. Obtener el n-ésimo bit de una variable (bit_get).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n)          (1<<(n))  
#define BIT_GET(x,n)   ((x) & BIT(n))
```

1. Obtener el n-ésimo bit de una variable (bit_get).

Supongamos $x = 109_{10} = 01101101_2$

```
#define BIT(n) (1<<(n))
```

n	(1<<(n))	BIT(n)
0	00000001 ₂	1
1	00000010 ₂	2
2	00000100 ₂	4
3	00001000 ₂	8
4	00010000 ₂	16
5	00100000 ₂	32
6	01000000 ₂	64
7	10000000 ₂	128

```
#define BIT_GET(x,n) ((x) & BIT(n))
```

x	n	((x) & BIT(n))	BIT_GET(x,n)
109	0	00000001 ₂	1
109	1	00000000 ₂	0
109	2	00000100 ₂	4
109	3	00001000 ₂	8
109	4	00000000 ₂	0
109	5	00100000 ₂	32
109	6	01000000 ₂	64
109	7	00000000 ₂	0

```
#define BIT_SET(x,b) ((x) |= BIT(b))  
#define BIT(n) (1<<(n))
```

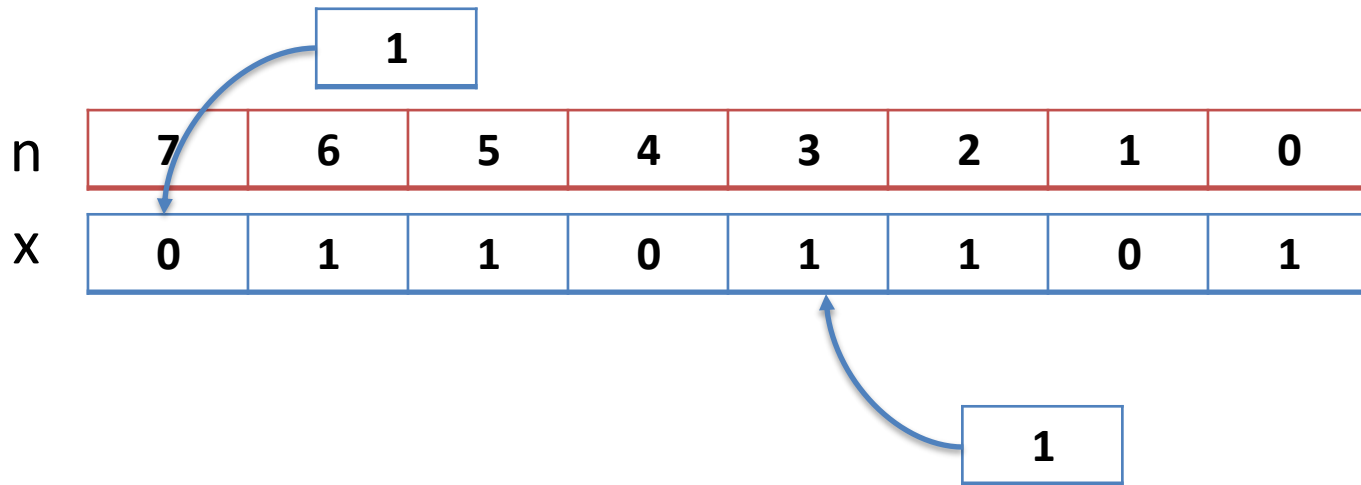
MANIPULACIÓN DE BITS / Operaciones básica



UNIVERSIDAD DE
GUANAJUATO

2. Establecer a '1' el n-ésimo bit de una variable (bit_set).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n) (1<<(n))  
#define BIT_SET(x,n) ((x) |= BIT(n))
```

2. Establecer a '1' el n-ésimo bit de una variable (bit_set).

Supongamos $x = 109_{10} = 01101101_2$

#define BIT(n) (1<<(n))

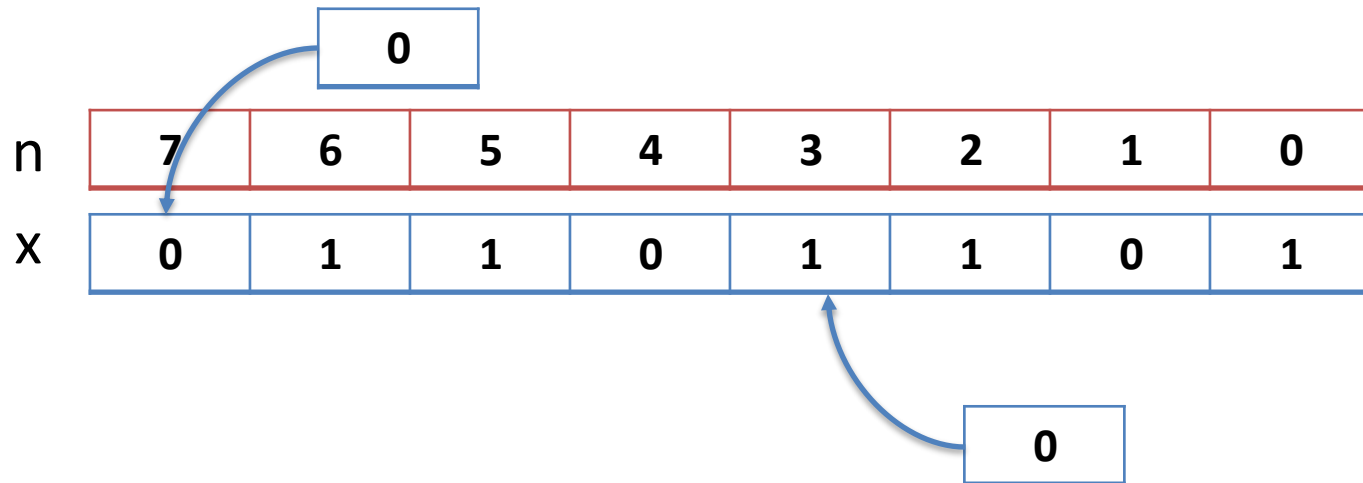
n	(1<<(n))	BIT(n)
0	00000001 ₂	1
1	00000010 ₂	2
2	00000100 ₂	4
3	00001000 ₂	8
4	00010000 ₂	16
5	00100000 ₂	32
6	01000000 ₂	64
7	10000000 ₂	128

#define BIT_SET(x,n) ((x) |= BIT(n))

x	n	((x) = BIT(n))	BIT_SET(x,n)
109	0	01101101 ₂	109
109	1	01101111 ₂	111
109	2	01101101 ₂	109
109	3	01101101 ₂	109
109	4	01111101 ₂	125
109	5	01101101 ₂	109
109	6	01101101 ₂	109
109	7	11101101 ₂	237

3. Establecer a '0' el n-ésimo bit de una variable (bit_clear).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n)          (1<<(n))
#define BIT_CLEAR(x,n) ((x) &= ~BIT(n))
```

3. Establecer a '0' el n-ésimo bit de una variable (bit_clear).

Supongamos $x = 109_{10} = 01101101_2$

#define BIT(n) (1<<(n))

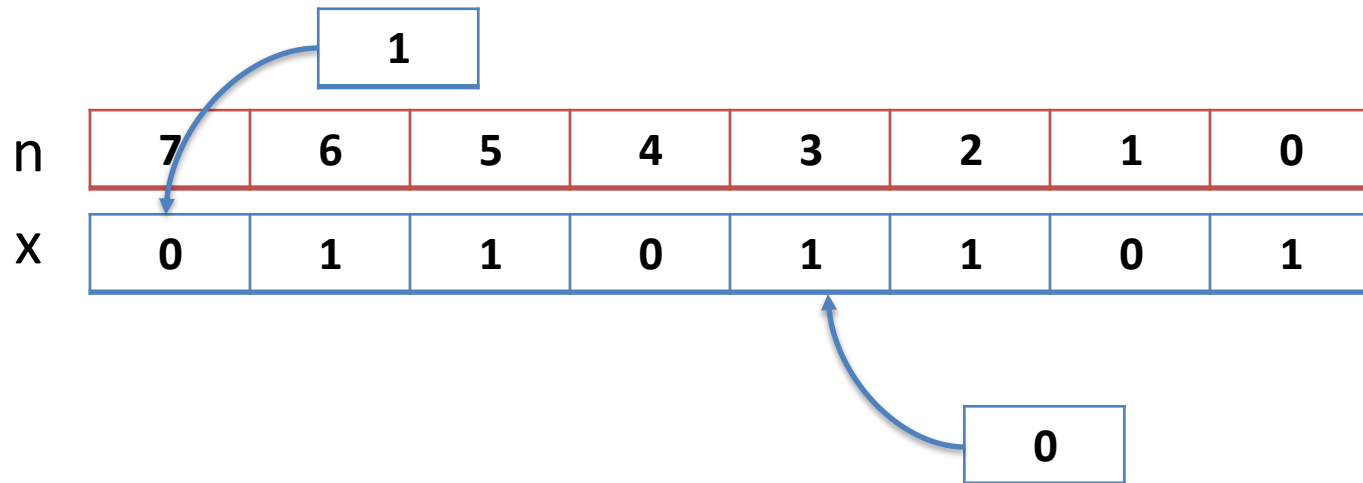
n	(1<<(n))	BIT(n)
0	00000001 ₂	1
1	00000010 ₂	2
2	00000100 ₂	4
3	00001000 ₂	8
4	00010000 ₂	16
5	00100000 ₂	32
6	01000000 ₂	64
7	10000000 ₂	128

#define BIT_CLEAR(x,n) ((x) &= ~BIT(n))

x	n	((x) &= ~BIT(n))	BIT_CLEAR(x,n)
109	0	01101100 ₂	108
109	1	01101101 ₂	109
109	2	01101001 ₂	105
109	3	01100101 ₂	93
109	4	01101101 ₂	109
109	5	01001101 ₂	77
109	6	00101101 ₂	45
109	7	01101101 ₂	109

4. Invertir el valor del n-ésimo bit de una variable (bit_toggle).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n)          (1<<(n))
#define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
```

4. Invertir el valor del n-ésimo bit de una variable (bit_toggle).

Supongamos $x = 109_{10} = 01101101_2$

#define BIT(n) (1<<(n))

n	(1<<(n))	BIT(n)
0	00000001 ₂	1
1	00000010 ₂	2
2	00000100 ₂	4
3	00001000 ₂	8
4	00010000 ₂	16
5	00100000 ₂	32
6	01000000 ₂	64
7	10000000 ₂	128

#define BIT_TOGGLE(x,n) ((x) ^= BIT(n))

x	n	((x) ^= BIT(n))	BIT_TOGGLE(x,n)
109	0	01101100 ₂	108
109	1	01101111 ₂	111
109	2	01101001 ₂	105
109	3	01100101 ₂	93
109	4	01111101 ₂	141
109	5	01001101 ₂	77
109	6	00101101 ₂	45
109	7	11101101 ₂	237

```
#define BIT_SET(x,b) ((x) |= BIT(b))  
#define BIT_CLEAR(x,b) ((x) &= ~BIT(b))
```

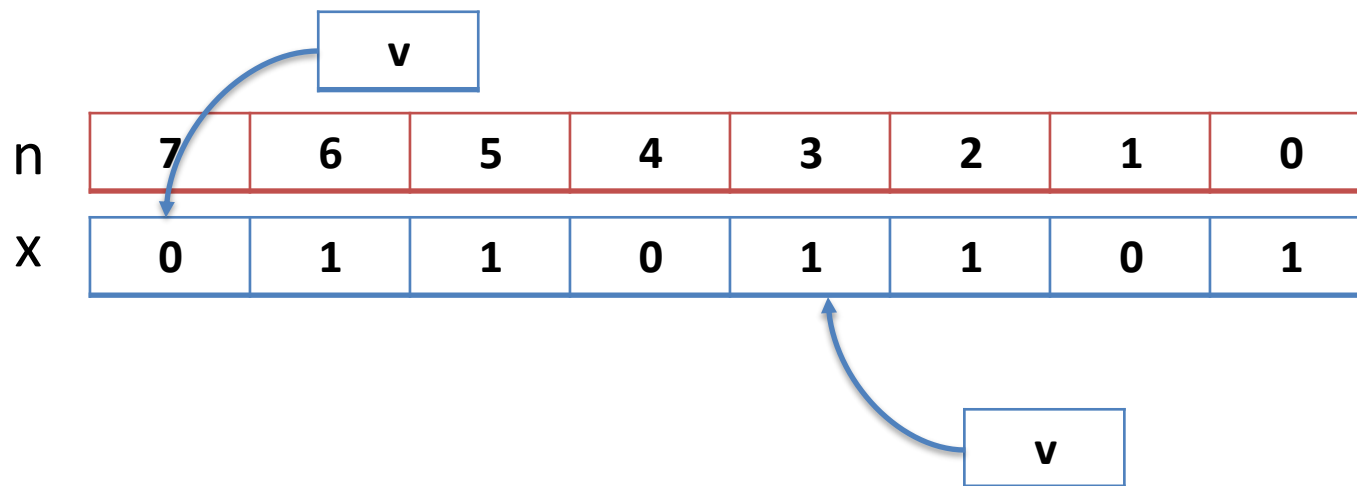
MANIPULACIÓN DE BITS / Operaciones básica



UNIVERSIDAD DE
GUANAJUATO

5. Establecer el valor 'v' en el n-ésimo bit de una variable (bit_write).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n) (1<<(n))  
#define BIT_WRITE(x,n,v) ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
```

5. Establecer el valor 'v' en el n-ésimo bit de una variable (bit_write).

Supongamos $x = 109_{10} = 01101101_2$

#define BIT(n) (1<<(n))

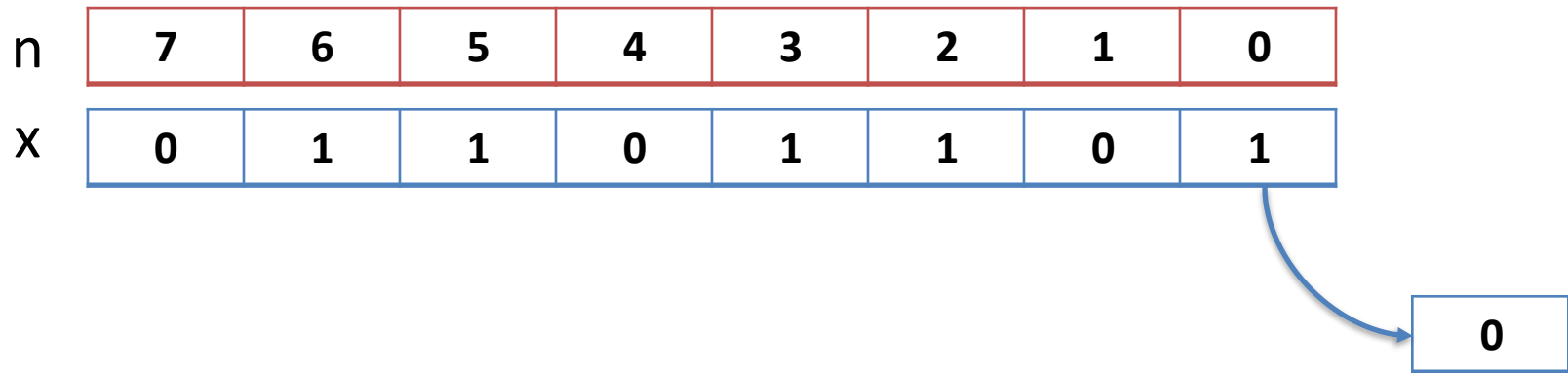
n	(1<<(n))	BIT(n)
0	00000001 ₂	1
1	00000010 ₂	2
2	00000100 ₂	4
3	00001000 ₂	8
4	00010000 ₂	16
5	00100000 ₂	32
6	01000000 ₂	64
7	10000000 ₂	128

#define BIT_WRITE(x,n,v) ((v)?BIT_SET(x,n):BIT_CLEAR(x,b))

x	n	V	((v)?BIT_SET(x,n): BIT_CLEAR(x,b))	BIT_WRITE(x,n,v)
109	0	0	01101100 ₂	108
109	1	1	01101111 ₂	111
109	2	0	01101001 ₂	105
109	3	1	01101101 ₂	109
109	4	0	01101101 ₂	109
109	5	1	01101101 ₂	109
109	6	0	00101101 ₂	45
109	7	1	11101101 ₂	237

Detectar si una variable es par o impar ($\text{es_par}(x)$).

Supongamos $x = 109_{10} = 01101101_2$



```
#define BIT(n)          (1<<(n))  
#define BIT_GET(x,n)   ((x) & BIT(n))  
#define ES_PAR(x)      (!BIT_GET(x,0))
```

Detectar si una variable es par o impar (`es_par(x)`).

Supongamos $x = 109_{10} = 01101101_2$

```
#define BIT(0)          (1<<(0))
```

$$1_{10} = 00000001_2$$

```
#define BIT_GET(109,0)  ((109) & BIT(0))
```

$$01101101_2 \& 00000001_2 = 00000001_2 = 1_{10}$$

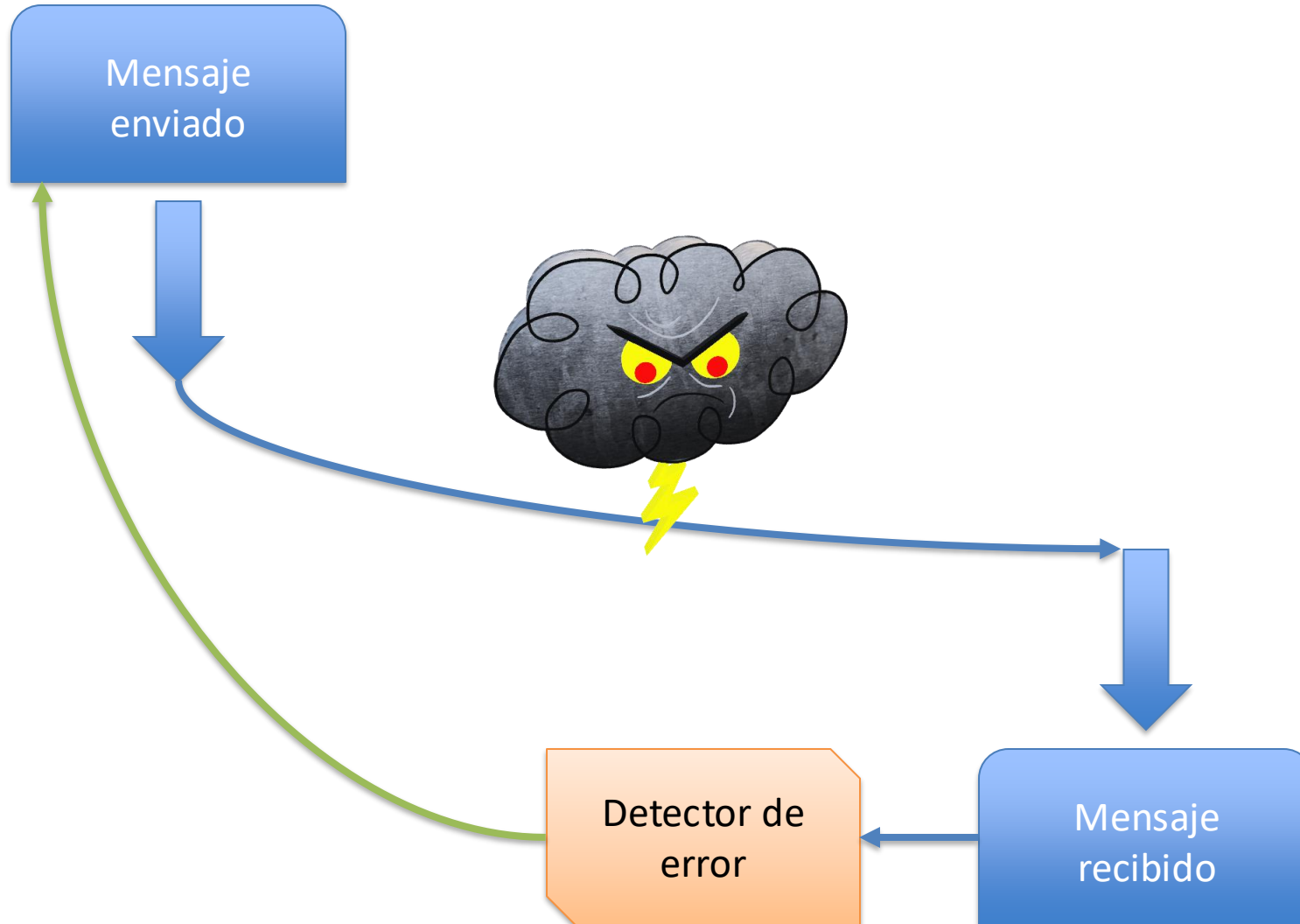
```
#define ES_PAR(109)     (!BIT_GET(109,0))
```

$$!(1_{10}) = 0_{10}$$

Detectar si una variable es par o impar (es_par(x)).

```
#define BIT(0)          (1<<(0))
                        110 = 000000012
```

x ₁₀	x ₂	((x) & BIT(0))	(!BIT_GET(x,0))
0	00000000	00000000	1
1	00000001	00000001	0
2	00000010	00000000	1
3	00000011	00000001	0
4	00000100	00000000	1
5	00000101	00000001	0
6	00000110	00000000	1
7	00000111	00000001	0
8	00001000	00000000	1
9	00001001	00000001	0
10	00001010	00000000	1



Generación de byte de paridad:

Supongamos un mensaje a enviar $msg[30] = \text{"Universidad de Guanajuato"}$

U	n	i	v	e	r	s	i	d	a	d		d	e
85	110	105	118	101	114	115	105	100	97	100	32	100	101
0101 0101	0110 1110	0110 1001	0111 0110	0110 0101	0111 0010	0111 0011	0110 1001	0110 0100	0110 0001	0110 0100	0010 0000	0110 0100	0110 0101
	G	u	a	n	a	j	u	a	t	o	/0		
32	71	117	97	110	97	106	117	97	116	111	0		
0010 0000	0100 0111	0111 0101	0110 0001	0110 1110	0110 0001	0110 1010	0111 0101	0110 0001	0111 0100	0110 1111	0000 0000		

$$n = \#\{msg\}$$

$$P = \bigotimes msg[i]$$

donde $\bigotimes$ es el operador XOR (^)

Generación de byte de paridad:

Supongamos un mensaje a enviar $msg[30] = \text{"Universidad de Guanajuato"}$

U	n	i	v	e	r	s	i	d	a	d		d	e
85	110	105	118	101	114	115	105	100	97	100	32	100	101
0101 0101	0110 1110	0110 1001	0111 0110	0110 0101	0111 0010	0111 0011	0110 1001	0110 0100	0110 0001	0110 0100	0010 0000	0110 0100	0110 0101
G	u	a	n	a	j	u	a	t	o	p	/0		
32	71	117	97	110	97	106	117	97	116	111	112	0	
0010 0000	0100 0111	0111 0101	0110 0001	0110 1110	0110 0001	0110 1010	0111 0101	0110 0001	0111 0100	0110 1111	0111 0000	0000 0000	

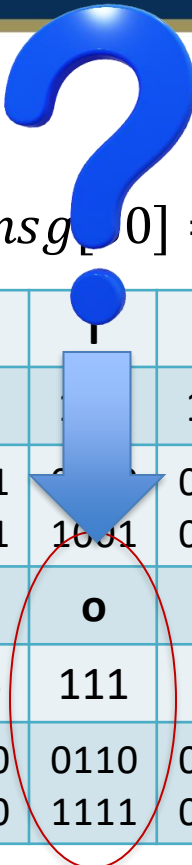
$$n_{paridad} = \#\{msg\} + 1$$

$$P = \bigotimes msg[i] = 01110000_2 = 112_{10}$$

$$V = (\bigotimes msg[i])^P = 00000000_2 = 0_{10}$$

Generación de byte de paridad:

Supongamos que el mensaje que llega $msg[0] = \text{"Universidad de Guanajuato"}$



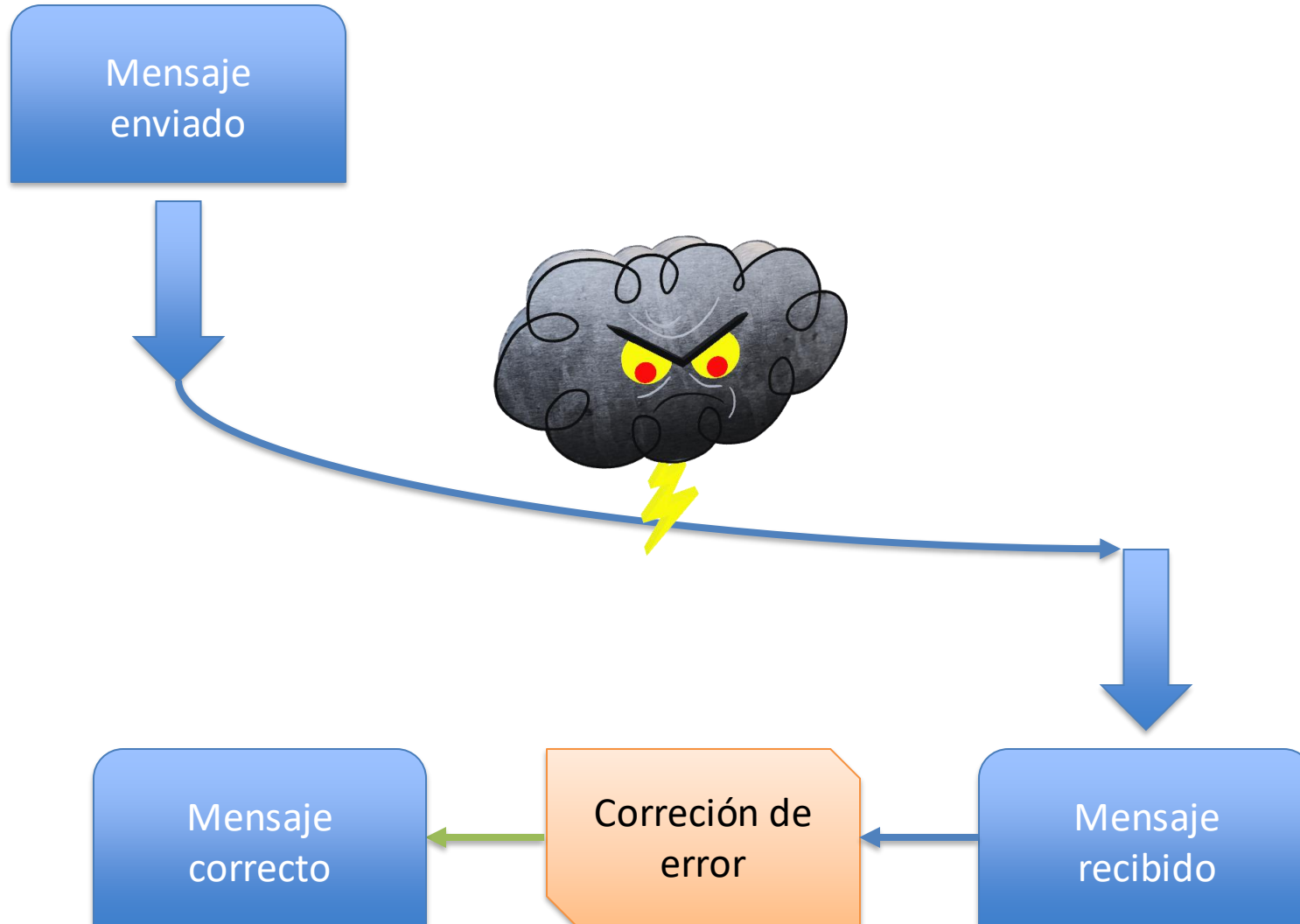
U	n	i	v	e	r	s	i	d	a	d		d	e
85	110	105	118	101	114	115	111	100	97	100	32	100	101
0101	0110	0110	0111	0110	0111	0111	0110	0110	0110	0110	0010	0110	0110
0101	1110	1001	0110	0101	0010	0011	1001	0100	0001	0100	0000	0100	0101
	G	u	a	n	a	j	o	a	t	o	p	/0	
32	71	117	97	110	97	106	111	97	116	111	112	0	
0010	0100	0111	0110	0110	0110	0110	0110	0110	0111	0110	0111	0000	
0000	0111	0101	0001	1110	0001	1010	1111	0001	0100	1111	0000	0000	

$$n_{paridad} = \#\{msg\} + 1$$

u	117	01110101
o	111	01101111
$u \wedge o$	26	00011010
$V \wedge o$	117	01110101

$$P = \bigotimes msg[i] = 01101010_2 = 106_{10}$$

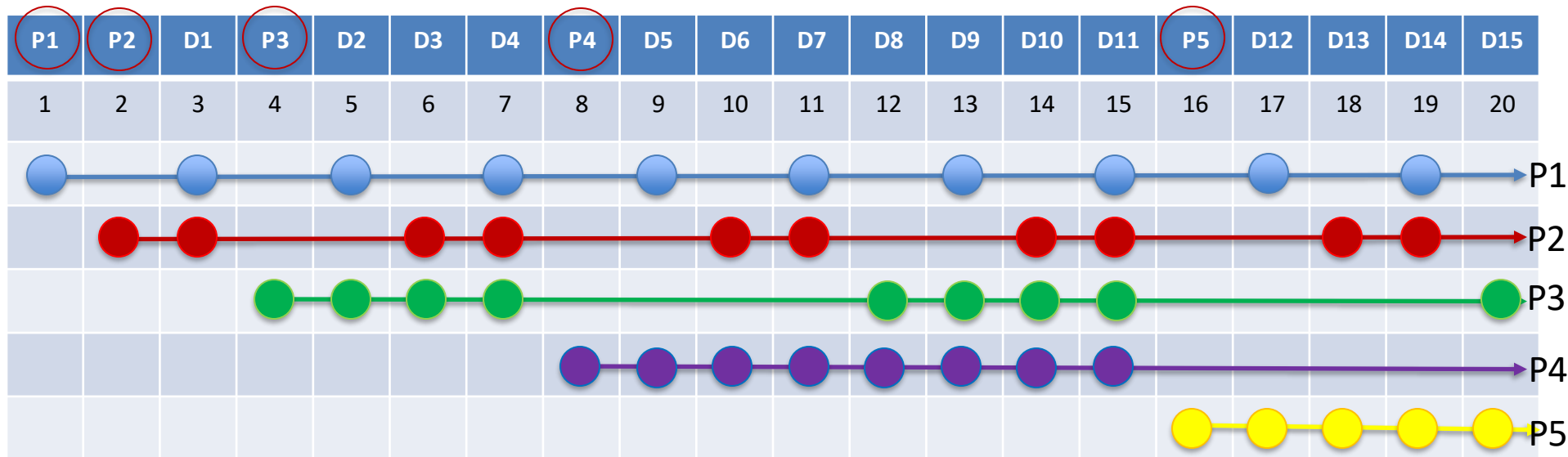
$$V = (\bigotimes msg[i]) \wedge P = 00011010_2 = 26_{10}$$



La corrección de errores mediante el código Hamming es un método deductivo para la detección y corrección de errores en mensajes.

En la implementación de este método es necesario considerar los siguientes puntos:

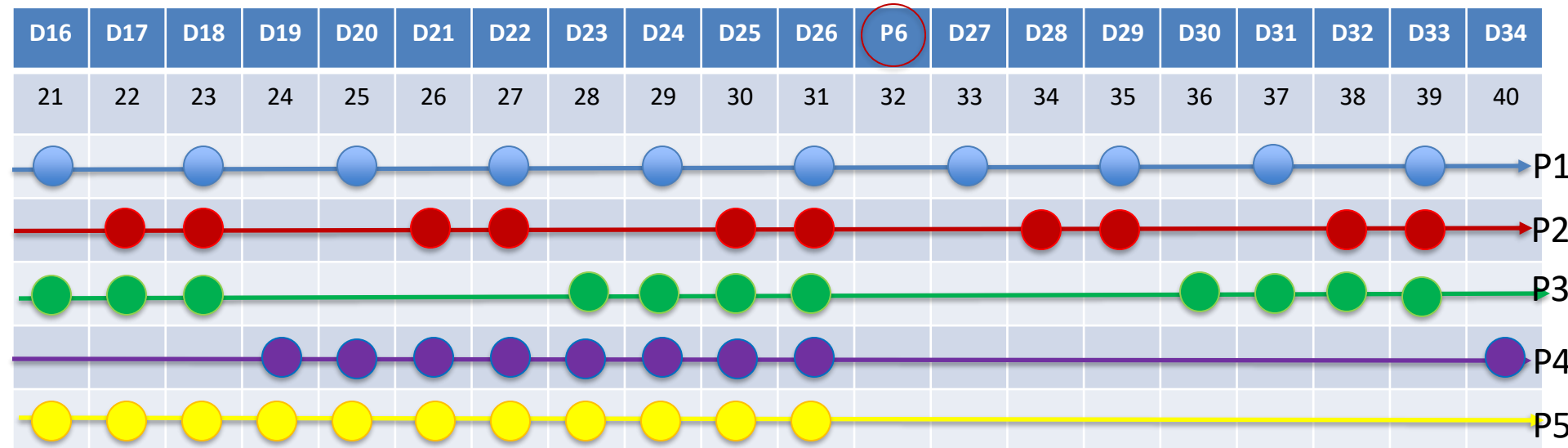
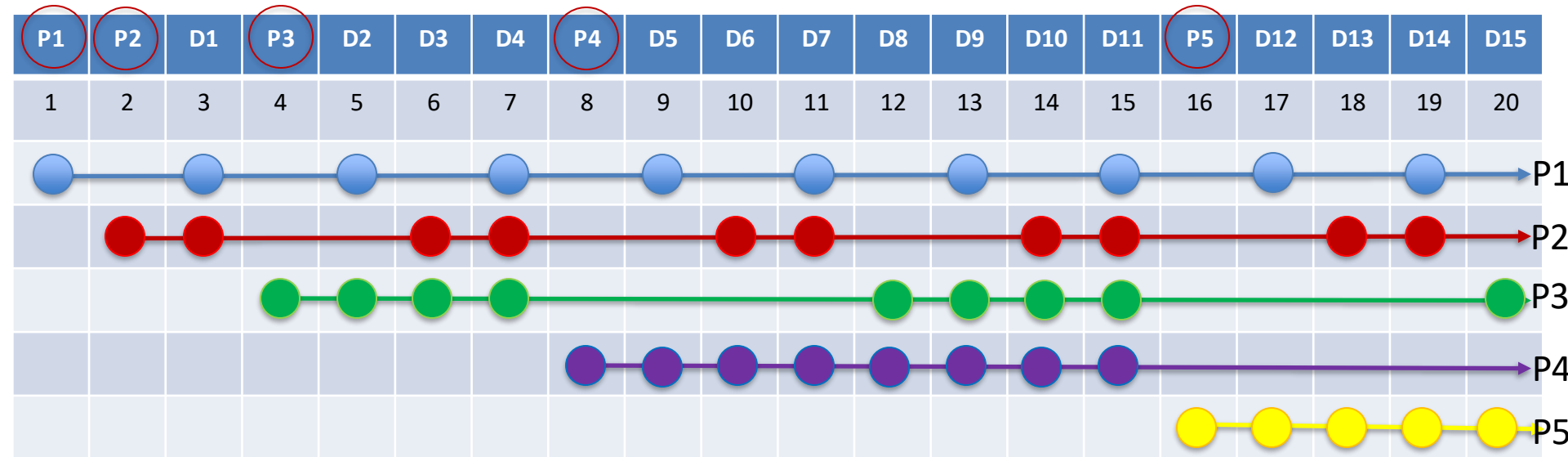
1. Utilizaremos bits de redundancia, mediante bits de paridad, para detectar si un mensaje en tiene errores y la posición del error.
2. Los bits de paridad estarán relacionados con determinados bits del mensaje enviado, esto nos permitirá posicionar el error en el mensaje recibido respecto del mensaje enviado. Los bits de paridad tendrán posiciones específicas en el mensaje, esto estarán en las posiciones correspondientes a las potencias de 2, es decir, 1, 2, 4, 8, 16, 32, ...
3. Los bit de paridad podada ser agregados por byte o por mensaje, según la aplicación.



MANIPULACIÓN DE BITS / Corrección de error (Hamming)



UNIVERSIDAD DE
GUANAJUATO



Supongamos que deseamos enviar el siguiente mensaje $msg[4] = \text{"Hola"}$

$$\text{"H"} = 72_{10} = 01001000_2$$

$$\text{"l"} = 108_{10} = 01101100_2$$

$$\text{"o"} = 111_{10} = 01101111_2$$

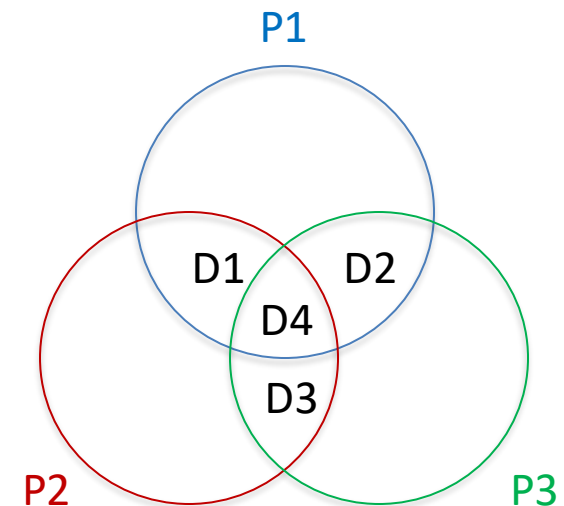
$$\text{"a"} = 97_{10} = 01100001_2$$

P1	P2	D1	P3	D2	D3	D4
		H		o	l	a
		0100 1000		0110 1111	0110 1100	0110 0001
0100 0110		0100 1000		0110 1111		0110 0001
	0100 0101	0100 1000			0110 1100	0110 0001
			0110 0010	0110 1111	0110 1100	0110 0001
0100 0110	0100 0101	0100 1000	0110 0010	0110 1111	0110 1100	0110 0001
F	E	H	b	o	l	a

$$P1 = \{D1, D2, D4\}$$

$$P2 = \{D1, D3, D4\}$$

$$P3 = \{D2, D3, D4\}$$





Supongamos que deseamos enviar el siguiente mensaje $msg[4] = \text{"Hola"}$

$$\text{"H"} = 72_{10} = 0100\ 1000_2$$

$$\text{"o"} = 111_{10} = 0110\ 1111_2$$

$$\text{"l"} = 108_{10} = 0110\ 1100_2$$

$$\text{"a"} = 97_{10} = 0110\ 0001_2$$

Utilizando el código Hamming enviaríamos el mensaje $msg[7] = \text{"FEHbola"}$

$$\text{"F"} = 70_{10} = 0100\ 0110_2$$

$$E = 69_{10} = 0100\ 0101_2$$

$$\text{"b"} = 98_{10} = 0110\ 0010_2$$

Supongamos un error en un carácter y que recibimos el mensaje $msg[7] = \text{"FEHbota"}$ verificando la paridad tendríamos.

$$\text{"t"} = 116_{10} = 0111\ 0100_2$$

$$V1 = F \oplus H \oplus o \oplus a = 0100\ 0110_2 \oplus 0100\ 1000_2 \oplus 0110\ 1111_2 \oplus 0110\ 0001_2 = 0000\ 0000_2$$

$$V2 = E \oplus H \oplus t \oplus a = 0100\ 0101_2 \oplus 0100\ 1000_2 \oplus 0111\ 0100_2 \oplus 0110\ 0001_2 = 0001\ 1000_2$$

$$V3 = b \oplus o \oplus t \oplus a = 0110\ 0010_2 \oplus 0110\ 1111_2 \oplus 0111\ 0100_2 \oplus 0110\ 0001_2 = 0001\ 1000_2$$

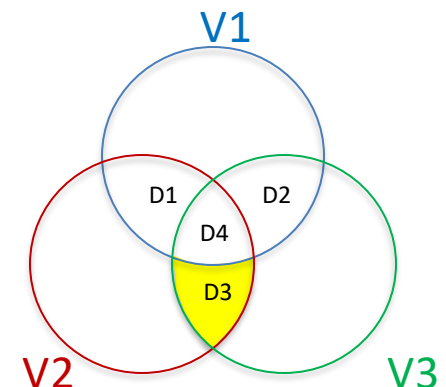
$$V = \emptyset \emptyset \emptyset D3\ D3\ \emptyset \emptyset \emptyset = 0001\ 1000_2$$

$$D3 = \text{"t"} = 116_{10} = 0111\ 0100_2$$

$$0111\ 0100_2 \oplus 0001\ 1000_2 = 0110\ 1100_2$$

$$D3 = 0110\ 1100_2 = 108_{10} = \text{"l"}$$

Mensaje corregido $msg[7] = \text{"FEHbola"}$ $msg[4] = \text{"Hola"}$





Supongamos que deseamos enviar el siguiente mensaje $msg[4] = \text{"Hola"}$

$$\text{"H"} = 72_{10} = 0100\ 1000_2$$

$$\text{"o"} = 111_{10} = 0110\ 1111_2$$

$$\text{"l"} = 108_{10} = 0110\ 1100_2$$

$$\text{"a"} = 97_{10} = 0110\ 0001_2$$

Utilizando el código Hamming enviaríamos el mensaje $msg[7] = \text{"FEHbola"}$

$$\text{"F"} = 70_{10} = 0100\ 0110_2$$

$$\text{E} = 69_{10} = 0100\ 0101_2$$

$$\text{"b"} = 98_{10} = 0110\ 0010_2$$

Supongamos varios error en diversos bits y que recibimos el mensaje $msg[7] = \text{"FEXbMle"}$ verificando la paridad tendríamos.

$$\text{"X"} = 88_{10} = 010\mathbf{1}\ 1000_2 \quad \text{"M"} = 77_{10} = 01\mathbf{00}\ 11\mathbf{01}_2 \quad \text{"e"} = 101_{10} = 0110\ 0\mathbf{1}01_2$$

$$V1 = F \oplus X \oplus M \oplus e = 0100\ 0110_2 \oplus 0101\ 1000_2 \oplus 0100\ 1101_2 \oplus 0110\ 0101_2 = 0011\ 0110_2$$

$$V2 = E \oplus X \oplus l \oplus e = 0100\ 0101_2 \oplus 0101\ 1000_2 \oplus 0110\ 1100_2 \oplus 0110\ 0101_2 = 0001\ 0100_2$$

$$V3 = b \oplus M \oplus l \oplus e = 0110\ 0010_2 \oplus 0100\ 1101_2 \oplus 0110\ 1100_2 \oplus 0110\ 0101_2 = 0010\ 0110_2$$

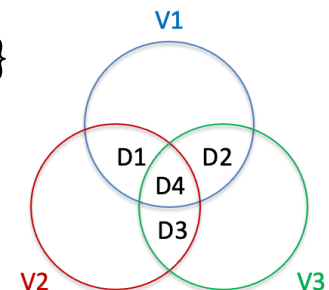
$$V = \emptyset \emptyset \mathbf{D2}\ \mathbf{D1}\ \emptyset\ \mathbf{D4}\ \mathbf{D2}\ \emptyset = \{0001\ 0000_2, \mathbf{0010\ 0010}_2, 0000\ 0100_2\}$$

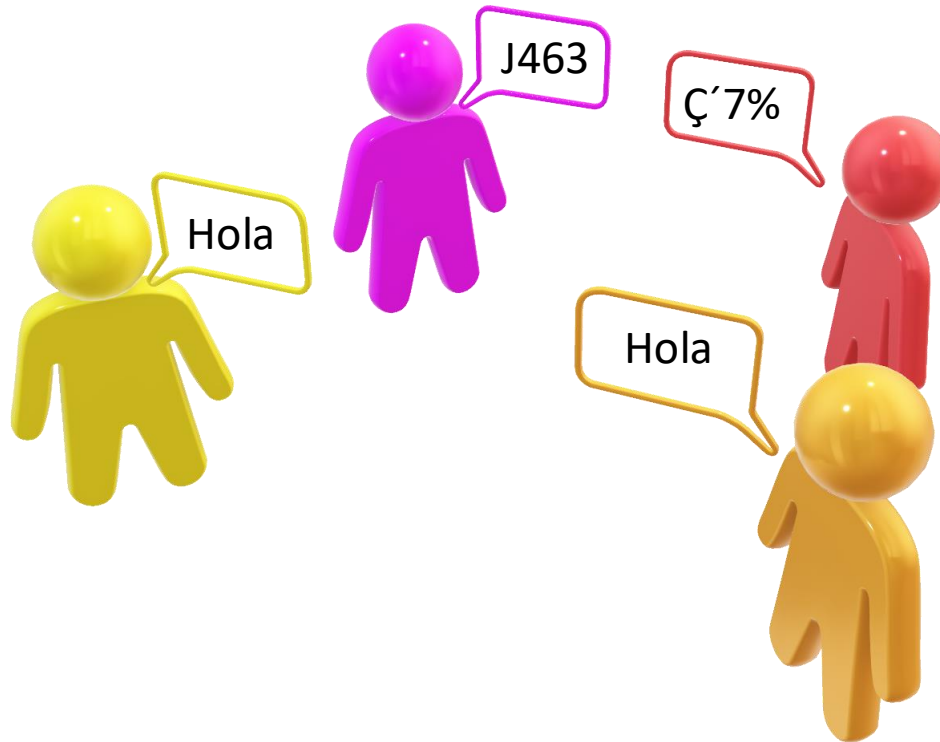
$$0101\ 1000_2 \oplus 0001\ 0000_2 = 0100\ 1000_2 = 72_{10} = \text{"H"}$$

$$0100\ 1101_2 \oplus \mathbf{0010\ 0010}_2 = 0110\ 1111_2 = 111_{10} = \text{"o"}$$

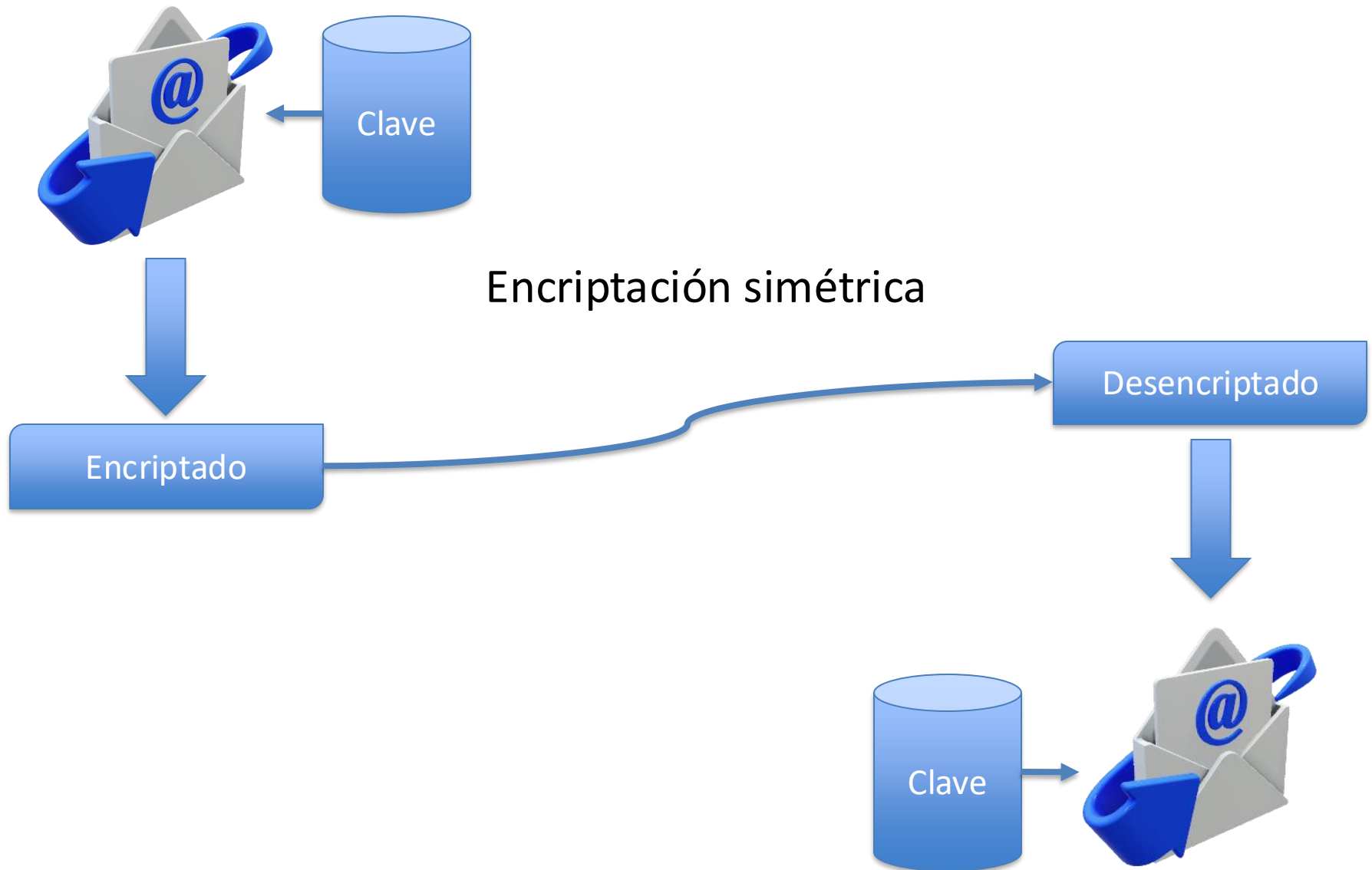
$$0110\ 0101_2 \oplus 0000\ 0100_2 = 0110\ 0001_2 = 97_{10} = \text{"a"}$$

Mensaje corregido $msg[7] = \text{"FEHbola"}$ $msg[4] = \text{"Hola"}$





La criptografía es el procedimiento que traduce el texto sin formato (texto plano) en una secuencia ilegible de caracteres mediante una clave. El objetivo es que el contenido del texto secreto resultante o criptograma (texto cifrado) solo sea accesible para aquellos que disponen de la clave para descifrarlo. Los métodos criptográficos son aplicados a cualquier tipo de información electrónica como mensajes de voz, archivos de imagen o códigos de programación, además de a mensajes de texto.





“Hola”

72₁₀ 111₁₀ 108₁₀ 97₁₀

0100 1000₂

0110 1111₂

0110 1100₂

0110 0001₂

0100 0100₂

0110 0011₂

0110 0000₂

0110 1101₂

68₁₀ 99₁₀ 96₁₀ 109₁₀

“Dc`m”

Mensaje original

Operador XOR

Clave

Mensaje encriptado

12₁₀

0000 1100₂

0100 1000₂

0110 1111₂

0110 1100₂

0110 0001₂

72₁₀ 111₁₀ 108₁₀ 97₁₀

“Hola”

Mensaje original

Operador XOR

Clave

Mensaje encriptado

12₁₀

0000 1100₂

68₁₀ 99₁₀ 96₁₀ 109₁₀

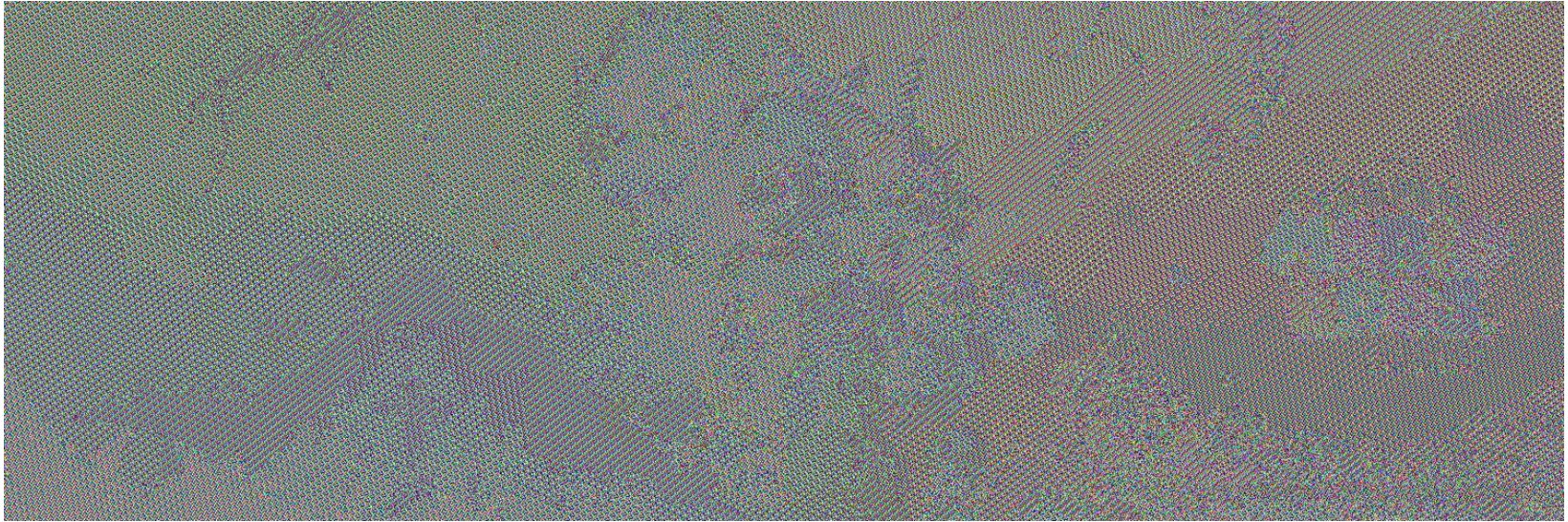
“Dc`m”

0100 0100₂

0110 0011₂

0110 0000₂

0110 1101₂



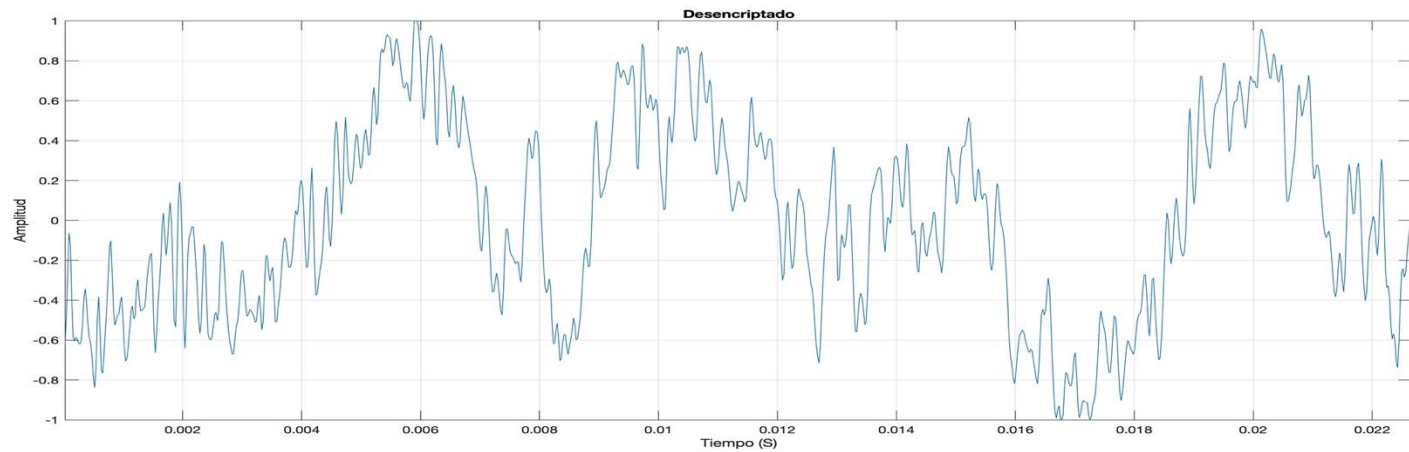
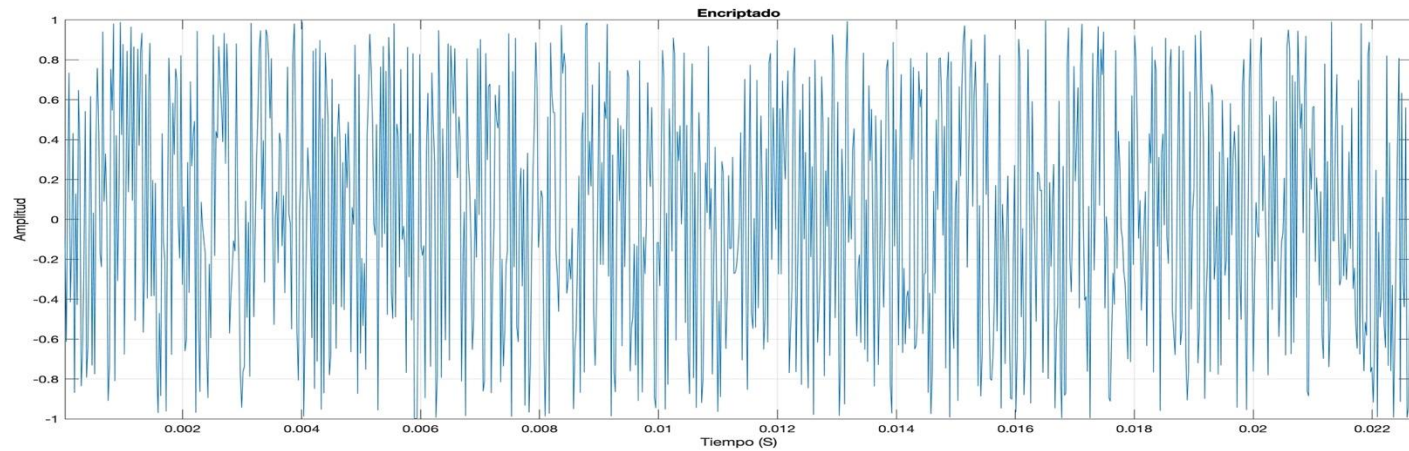
Clave = [229 198 184 43 51 87 245 217 56 40 223 44 133 116 83 18 174 178 46 210
183 20 192 106 221 103 5 209 244 75 65 80 168 207 151 177 217 226 6]



MANIPULACIÓN DE BITS / Encriptación de información



UNIVERSIDAD DE
GUANAJUATO



$$[A, Z] = [65, 90] \quad n_1 = \# = 26$$

$$[a, z] = [97, 122] \quad n_2 = \# = 26$$

$$n = n_1 + n_2 = 52$$

$$\min = \begin{cases} 65 & 65 \leq str_i \leq 90 \\ 97 & 97 \leq str_i \leq 122 \end{cases}$$

$$k = \begin{cases} 0 & 65 \leq str_i \leq 90 \\ 1 & 97 \leq str_i \leq 122 \end{cases}$$

$$str = "Hola" = [72 \ 111 \ 108 \ 97]$$

$$m_i = str_i - \min + k \times n_1$$

$$m_0 = 72 - 65 + 0 \times 26 = 7$$

$$m_1 = 111 - 97 + 1 \times 26 = 40$$

$$m_2 = 108 - 97 + 1 \times 26 = 37$$

$$m_3 = 97 - 97 + 1 \times 26 = 26$$

$$m = [7 \ 40 \ 37 \ 26]$$

$$m = [7 \ 40 \ 37 \ 26]$$

$$offset = \begin{cases} 65 & m_i \leq 25 \\ 71 & m_i > 25 \end{cases}$$

$$str_i = m_i + offset$$

$$str_0 = 7 + 65 = 72 = "H"$$

$$str_1 = 40 + 71 = 111 = "o"$$

$$str_2 = 37 + 71 = 108 = "l"$$

$$str_3 = 26 + 71 = 97 = "a"$$

$$str = [72 \ 111 \ 108 \ 97]$$

$$str = "Hola"$$

n número de símbolos
 $0 < a < n$ $\gcd(a, n) = 1$
 $0 \leq b \leq n$

$[A, Z] = [65, 90]$ $n_1 = \#[A, Z] = 26$
 $[a, b] = [97, 122]$ $n_2 = \#[a, b] = 26$
 $n_1 = n_2 = 26$

$\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$

$n = n_1 + n_2 = 52$

$\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25, 27\}$
 $\{29, 31, 33, 35, 37, 41, 43, 45, 47, 49, 51\}$

"Hola"

[72 111 108 97]

[7 40 37 26]

$c_i = (a \times m_i + b) \% n$

$c_0 = (17 \times 7 + 21) \% 52 = 36$

$c_1 = (17 \times 40 + 21) \% 52 = 25$

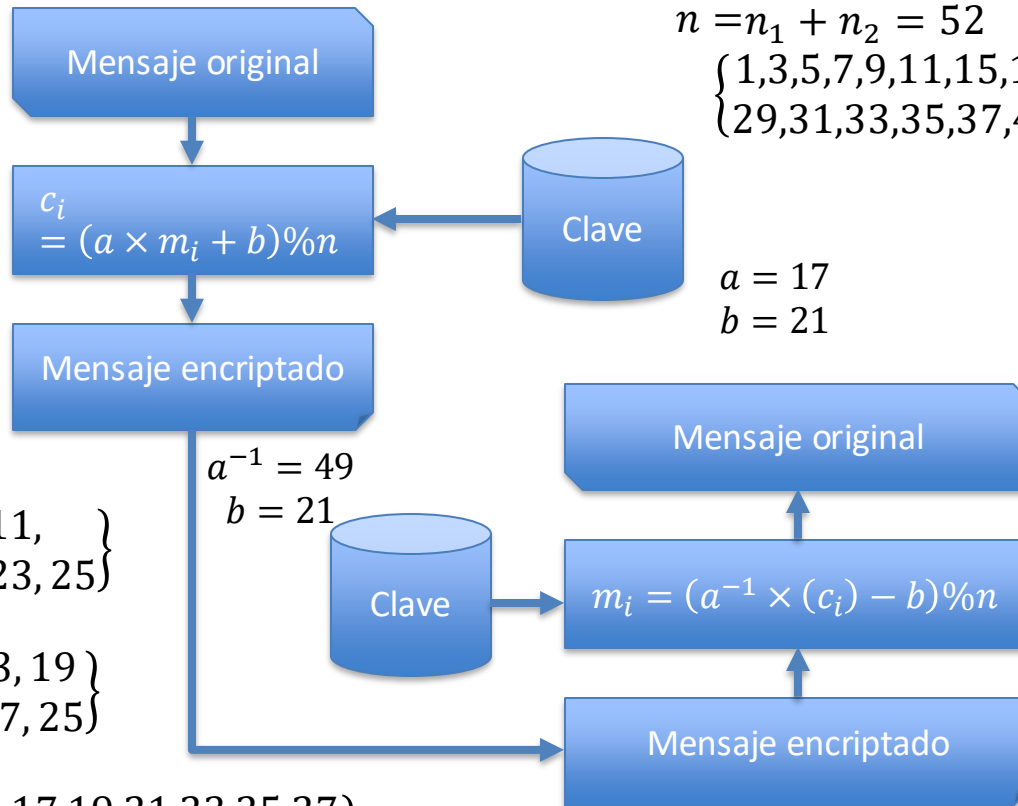
$c_2 = (17 \times 37 + 21) \% 52 = 26$

$c_3 = (17 \times 26 + 21) \% 52 = 47$

[36 25 26 47]

[107 90 97 118]

"kZav"



$a = 17$
 $b = 21$

$(a \times a^{-1}) \% n = 1$

$a = \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$

$a^{-1} = \{1, 9, 21, 15, 3, 19, 7, 23, 11, 5, 17, 25\}$

$a = \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 41, 43, 45, 47, 49, 51\}$

$a^{-1} = \{1, 35, 21, 15, 29, 19, 7, 49, 11, 5, 43, 25, 27, 9, 47, 41, 3, 45, 33, 23, 37, 31, 17, 51\}$

Mensaje original

$m_i = (a^{-1} \times (c_i - b)) \% n$

Mensaje encriptado

"kZav"

[107 90 97 118]

[36 25 26 47]

$m_i = (a^{-1} \times (c_i - b)) \% n$

$m_0 = (49 \times (36 - 21)) \% 52 = 7$

$m_1 = (49 \times (25 - 21)) \% 52 = 40$

$m_2 = (49 \times (26 - 21)) \% 52 = 37$

$m_3 = (49 \times (47 - 21)) \% 52 = 26$

[7 40 37 26]

[72 111 108 97]

"Hola"

$$\gcd(a, n) = 1$$

$$\gcd(a, 52) = 1$$

$$\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25, 27\}$$

$$\{29, 31, 33, 35, 37, 41, 43, 45, 47, 49, 51\}$$

$$a = \left\{ \begin{array}{l} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 \\ 15, 16, 17, 18, 19, 20, 21, 22, 23, 24 \\ 25, 26, 27, 28, 29, 30, 31, 32, 33, 34 \\ 35, 36, 37, 38, 39, 40, 41, 42, 43, 44 \\ 45, 46, 47, 48, 49, 50, 51 \end{array} \right\}$$

$$\text{factor}(52) = 2 \times 2 \times 13$$

$$\text{factor}(52) = \{2, 4, 13, 26\}$$

$$a \% 2! = 0$$

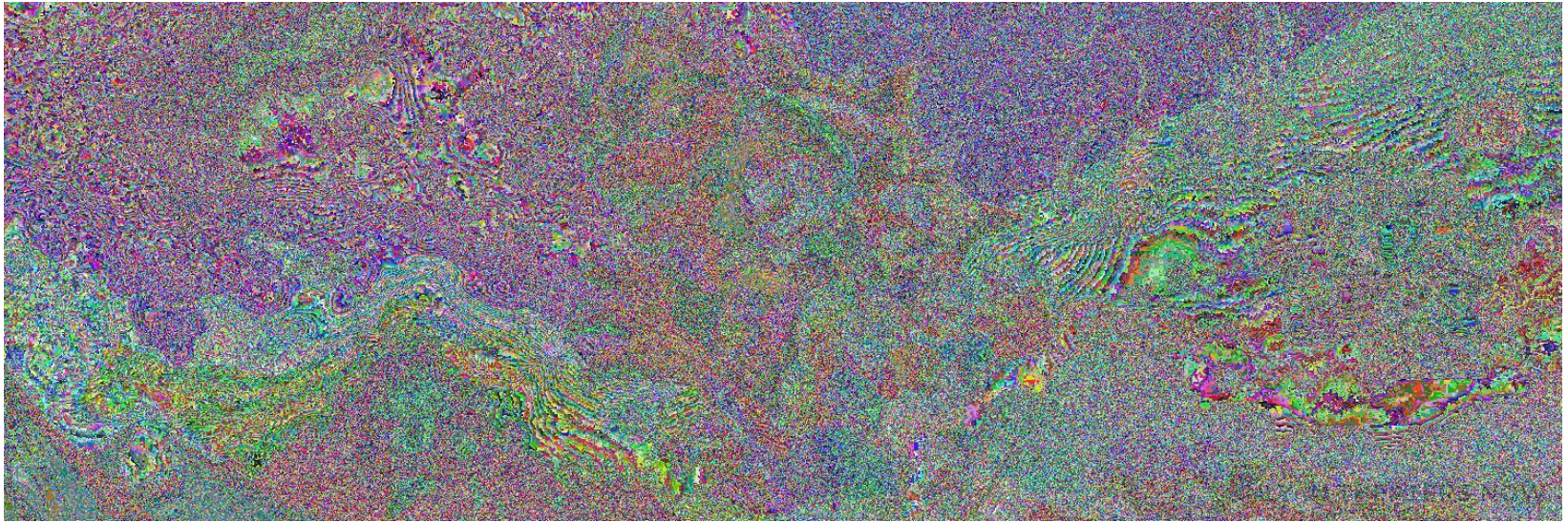
$$a \% 13! = 0$$

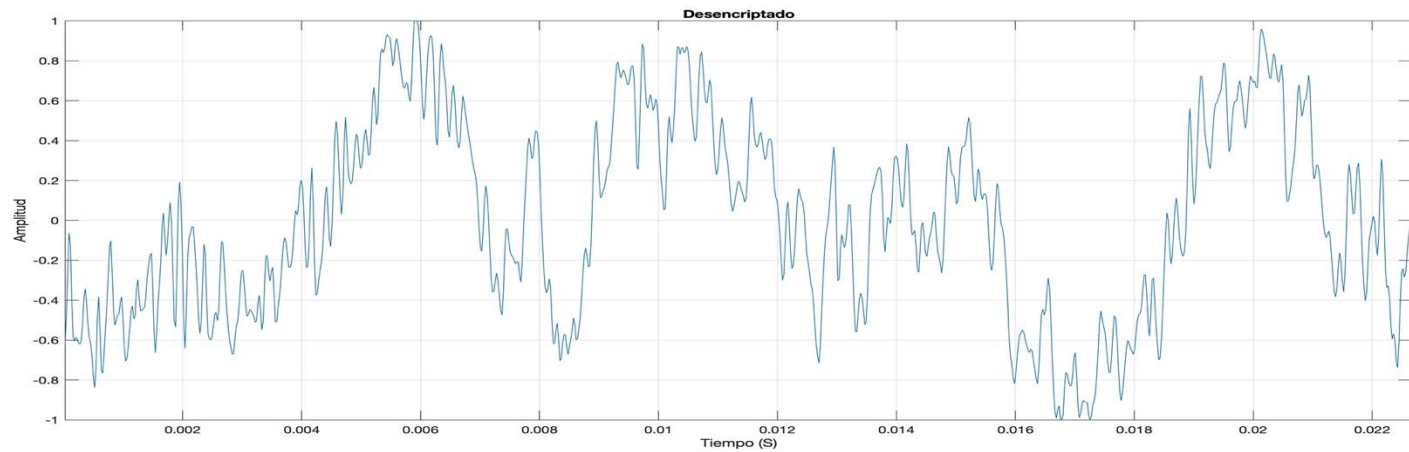
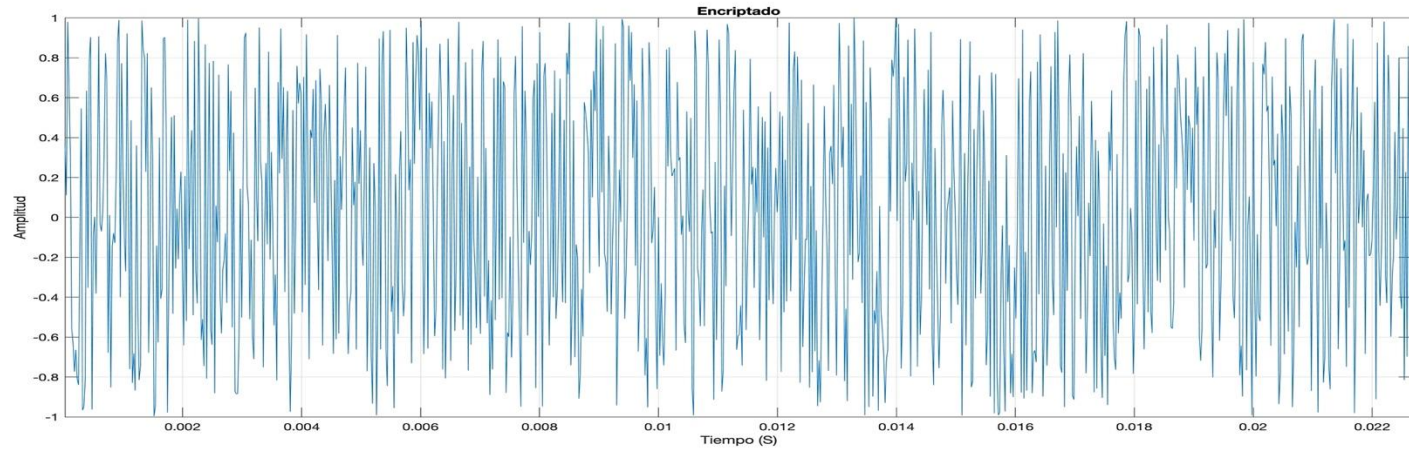
$$0 < a < n \quad \gcd(a, 52) = 1$$

$$0 < a \ \& \ a < n \ \& \ a \% 2! = 0 \ \& \ a \% 13! = 0$$

$$0 \geq a || a \geq n || a \% 2 == 0 || a \% 13 == 0$$

$$1 > a || a \geq n || a \% 2 == 0 || a \% 13 == 0$$





UNIVERSIDAD DE GUANAJUATO

